Inheritance & Polymorphism — The Two Pillars That Let Classes Talk to Each Other

Object-Oriented Programming — Lecture 4

Formatted by: Heisenberg 😎

Index

- 1. Setting the Stage — a quick reunion with objects, classes, and the four OOP pillars before we zoom into two of them.
- 2. Inheritance — Letting One Class Borrow Another's Toolbox — because reinventing the wheel is for people who don't have `extends`.
- 3. Polymorphism — One Name, Many Faces — same method name, identity crisis optional.
- 4. A Full Walkthrough: The Pet Family — Pet, Cat, and Dog teach us subclasses, overriding, and why a Dog can't pretend to be a generic Pet when it wants to fetch.
- 5. Defining Classes with Inheritance: The Student Roster Problem — grading undergrads and grad students without writing the same code twice.
- 6. Using Classes Effectively with Polymorphism: The Roster Array — one array, many student "flavours", zero type-checking headaches.
- 7. Summary — the whole lecture in one paragraph, for the night before the exam.
- 8. Practice Questions — five questions to make sure this actually stuck.

---

1. Setting the Stage

Before diving into today's topic, it helps to remember the four pillars of Object-Oriented Programming that keep showing up in every OOP course: **Abstraction**, **Encapsulation**, **Inheritance**, and **Polymorphism**. Earlier classes already built the foundation — objects, methods, classes, and the idea of single vs. multiple classes. Today's lecture picks up two of those four pillars — **Inheritance** and **Polymorphism** — and goes into real depth: what they are, how they are implemented in code, and what rules govern their correct use.

Keep three ideas in your back pocket as you read on, because everything below is built on top of them:

- **Object** — a concrete instance of a class.
- **Method** — a behavior defined inside a class.
- **Class** — the blueprint that defines both data (fields) and behavior (methods).

2. Inheritance — Letting One Class Borrow Another's Toolbox

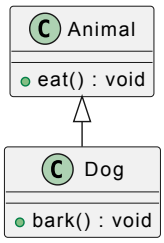
Inheritance allows one class to inherit the properties of another class. Instead of writing every class from scratch, a new class can simply *extend* an existing one and automatically gain access to its fields and methods.

Consider two classes: `Animal` and `Dog`.

```
class Animal {
    void eat() {
        System.out.println("Eating");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Barking");
    }
}
```

Here, `Dog` uses the keyword `extends` to inherit from `Animal`. This single keyword is doing a lot of work — it tells the compiler "everything `Animal` can do, `Dog` can do too, plus whatever `Dog` adds on its own."



If we now create a `Dog` object and call both methods on it, the output is simply:

```
Eating
Barking
```

Even though `eat()` was never written inside `Dog`, it is fully accessible — that is the entire point of inheritance: the child class inherits *both* the data members and the methods of the parent class.

3. Polymorphism — One Name, Many Faces

If inheritance is about *one class handing down its toolbox to another*, polymorphism is about *one interface, many implementations*. The clearest way to see this is method **overloading** — multiple methods sharing the same name inside the *same* class, but differing in their return type and/or argument list.

```
class MathDemo {
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }
}
```

Both methods are named `add`, they live in the same class `MathDemo`, yet their return types and parameter types are different. The compiler figures out *which* `add` to call based on the arguments you pass in — that is the essence of polymorphism: **a single name, multiple behaviors.**

> **Note for Review:** This particular flavor of polymorphism (same name, different parameter signatures, resolved at compile time) is called **overloading**. Later in the chapter, when a subclass redefines a method it inherited (same name, same signature), that's called **overriding**, and it is resolved at *runtime*. Both are polymorphism, but examiners love asking you to tell them apart — overloading = "many methods, same class, different signatures"; overriding = "same method, different classes, same signature."

4. A Full Walkthrough: The Pet Family

To see inheritance and polymorphism working together, we build a small hierarchy from scratch, starting with the most generic possible pet.

4.1 The Pet Class — A Simple Blueprint

The class is deliberately kept simple (no constructor) so the focus stays purely on the inheritance mechanism.

```
class Pet {
    private String name;

    public String getName() {
        return name;
    }

    public void setName(String petName) {
        name = petName;
    }

    public String speak() {
        return "I'm your cuddly little pet.";
    }
}
```

And here is sample code that uses this class:

```
Pet myPet = new Pet();
System.out.println(myPet.speak());
```

`myPet` is an **object of the `Pet` class**, created using `new Pet()`. Through this object, any public method — including `speak()` — can be called.

There are, of course, many different kinds of pets, and it would be unreasonable to expect a single generic `Pet` class to model a dog, a cat, and a reptile equally well. Dogs, cats, and reptiles are all *different*, yet they share the common trait of being pets. This is exactly the situation inheritance was built for: rather than defining `Cat` and `Dog` completely independently, we derive them from `Pet` so they automatically share its common data members and methods.

4.2 Subclass, Superclass, and Overriding — the Cat Class

```
class Cat extends Pet {
    public String speak() {
        return "Don't give me orders.\n" +
               "I speak only when I want to.";
    }
}
```

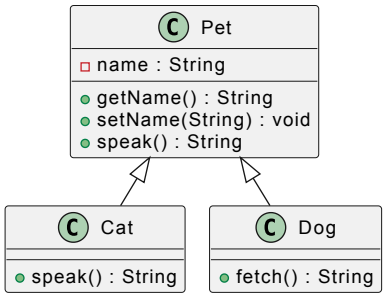
The keyword `extends` tells us that `Cat` is a **subclass** (also called a *derived class*) of `Pet`, and `Pet` is the **superclass** (also called the *base class*) of `Cat`. All data members and methods of the superclass are inherited by the subclass, so the following is perfectly valid even though `setName` and `getName` were never written inside `Cat`:

```
Cat myCat = new Cat();
myCat.setName("Cha Cha");
System.out.println("Hi, my name is " + myCat.getName());
```

Notice, though, that `Cat` redefines `speak()` with a completely different body. This is called **overriding** — the `Cat` class overrides the `speak()` method it inherited from `Pet`. So this code:

```
Cat myCat = new Cat();
myCat.setName("Puff Puff");
System.out.println(myCat.getName() + " says: ");
System.out.println(myCat.speak());
```

prints the *overridden* version of `speak()` (the sassy cat one), not the generic "cuddly little pet" line from `Pet`.



4.3 Adding New Behavior — the Dog Class

A subclass isn't limited to overriding — it can also add brand-new methods and data members that simply don't exist in the superclass:

```
class Dog extends Pet {
    public String fetch() {
        return "Yes, master. Fetch I will.";
    }
}
```

`fetch()` is unique to `Dog`; it doesn't exist anywhere in `Pet` and won't be inherited by `Cat`.

4.4 Results: What Actually Prints

```
Dog myDog = new Dog();
myDog.setName("Fifi");

System.out.println(myCat.getName() + " says: ");
System.out.println(myDog.speak());
System.out.println(myDog.fetch());
```

```
Fifi says:
I'm your cuddly little pet.
Yes, master. Fetch I will.
```

Walk through why: `myDog.speak()` isn't overridden inside `Dog`, so it falls back to the inherited version from `Pet` ("I'm your cuddly little pet"). `myDog.fetch()`, on the other hand, is defined right there in `Dog`, so it prints its own message.

4.5 The Golden Rule of References: Up Is Fine, Down Is Not

Now consider this code:

```
Pet petOne = new Dog();
Pet petTwo = new Cat();
```

Will it work? Yes. When a variable is declared to be of class `S` (here, `Pet`), that variable can reference an instance of `S` *or any of its subclasses* (here, `Dog` and `Cat`). The reverse, however, is **not** valid:

```
Dog myDog = new Pet();   // INVALID
```

You cannot create a `Dog` reference and point it at a plain `Pet` object, because a generic `Pet` doesn't know how to `fetch()` — the subclass reference promises capabilities the superclass object simply doesn't have. Every time, the right-hand side must be the same class or a more specific (sub)class than the left-hand side — never a more general one.

> **Note for Review:** Think of it like addressing an envelope. You can always address a package to "Animal" and put a "Dog" inside — the postman (compiler) is happy either way, because a Dog *is* an Animal. But you can't address a package specifically to "Dog" and put a generic "Animal" inside — the postman has no guarantee that whatever's in there actually knows how to bark.

4.6 Polymorphic Methods & the Need for Typecasting

The fact that the *same* variable can refer to instances of *different* classes results in polymorphism. The following two statements will produce different output depending on whether `p` currently refers to a `Dog` or a `Cat`:

```
Pet p;

p = new Dog();
System.out.println(p.speak());

p = new Cat();
System.out.println(p.speak());
```

Because `speak()` is overridden differently in each subclass, `p.speak()` behaves differently depending on what `p` is actually pointing to at runtime — this is why `speak()` is called a **polymorphic method**.

But polymorphism has a boundary. If a variable is declared of type `S` and refers to an instance of a subclass of `S`, then any method that is **unique to that subclass** (i.e., not present in `S`) cannot be called directly — the variable must first be **typecast** to the subclass. `fetch()` is defined only in `Dog`, so:

```
Pet p;
p = new Dog();
System.out.println(p.fetch());   // INVALID
```

is invalid, because as far as the compiler is concerned, `p` is a `Pet`, and `Pet` has no `fetch()` method. The fix is to explicitly typecast:

```
Pet p;
p = new Dog();
System.out.println(((Dog) p).fetch());   // valid
```

> **Note for Review:** Whenever you need to call a method that is unique to a subclass, and your variable's *declared* type is the superclass, typecasting is mandatory — no exceptions. This single rule trips up more students in exams than almost anything else in this chapter, so it's worth memorizing the pattern: `((SubclassName) referenceVariable).uniqueMethod()`.

Mastering inheritance and polymorphism effectively means mastering the many small rules that govern them — which is exactly what the rest of the chapter (and the worked example below) is about.

5. Defining Classes with Inheritance: The Student Roster Problem

Let's apply everything above to a more realistic scenario: maintaining a class roster where enrolled students can be either **undergraduate** or **graduate** students. For each student we record their name, three test scores, and a final course grade (**Pass** / **No Pass**), computed using the following rule:

Type of Student	Grading Scheme
Undergraduate	Pass if (test1 + test2 + test3) / 3 >= 70
Graduate	Pass if (test1 + test2 + test3) / 3 >= 80

Both types of students share almost everything — name, test scores, the ability to set/get them — and differ only in *how the passing threshold is computed*. This "mostly the same, slightly different" pattern is the textbook signal that inheritance is the right tool.

5.1 The `Student` Superclass

```
/*
   Chapter 13 Sample Program: Student
   File: Student.java
*/
class Student {

    protected final static int NUM_OF_TESTS = 3;

    protected String name;
    protected int[] test;
    protected String courseGrade;

    public Student() {
        this("No Name");
    }

    public Student(String studentName) {
        name = studentName;
        test = new int[NUM_OF_TESTS];
        courseGrade = "****";
    }

    public String getCourseGrade() {
        return courseGrade;
    }

    public String getName() {
        return name;
    }

    public int getTestScore(int testNumber) {
        return test[testNumber - 1];
    }

    public void setName(String newName) {
        name = newName;
    }

    public void setTestScore(int testNumber, int testScore) {
        test[testNumber - 1] = testScore;
    }

    public void computeCourseGrade() {
        // do nothing - override this method in the subclass
    }
}
```

A few details worth pausing on:

- `NUM_OF_TESTS` is declared `protected final static int` — it's a constant shared by every student, and it never changes once set.
- The fields `name`, `test`, and `courseGrade` are declared `protected` rather than `private`. **Protected fields are visible to descendant (subclass) objects** — this is what lets `GraduateStudent` and `UndergraduateStudent` reach directly into these fields when they compute the grade, something `private` would have blocked.
- `computeCourseGrade()` is deliberately left empty in the superclass — a placeholder that says "do nothing here; the real logic belongs in whichever subclass knows the actual grading rule." This is a common inheritance pattern: the superclass declares *that* the behavior exists, the subclass decides *how*.

> **Note for Review:** `private` vs `protected` vs `public` is really about "who's allowed to touch this." `private` = only this class. `protected` = this class *and* its subclasses. `public` = anyone. Inheritance hierarchies frequently use `protected` for exactly this reason — it lets subclasses collaborate with the superclass's internals without throwing the door wide open to the rest of the program.

5.2 Overriding `computeCourseGrade()` in Each Subclass

```
class GraduateStudent extends Student {

    public void computeCourseGrade() {
        int total = 0;
        for (int i = 0; i < NUM_OF_TESTS; i++) {
            total += test[i];
        }

        if (total / NUM_OF_TESTS >= 80) {
            courseGrade = "Pass";
        } else {
            courseGrade = "No Pass";
        }
    }
}
```

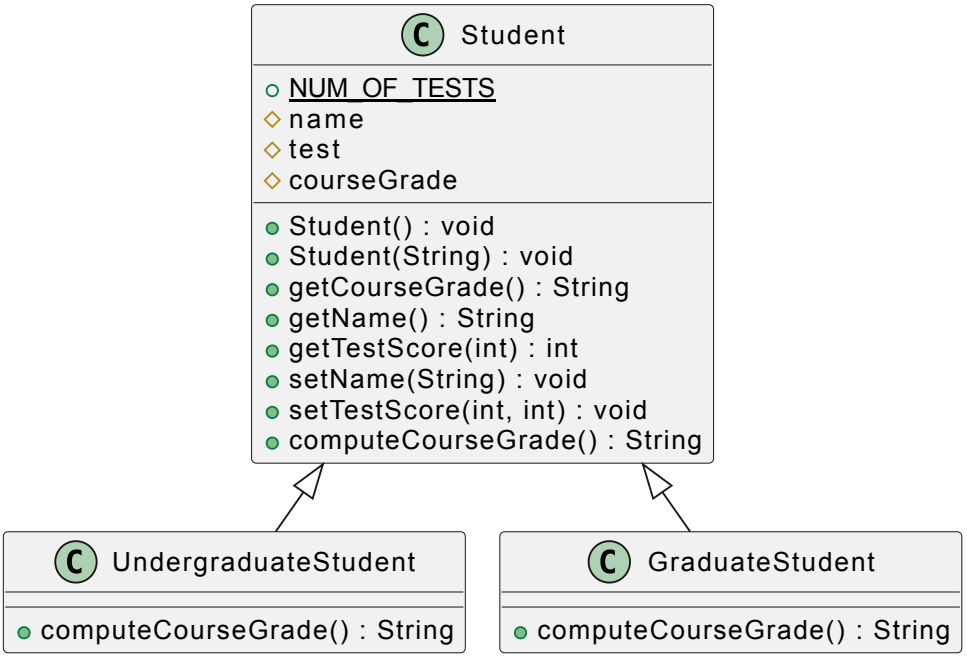
```
class UndergraduateStudent extends Student {

    public void computeCourseGrade() {
        int total = 0;
        for (int i = 0; i < NUM_OF_TESTS; i++) {
            total += test[i];
        }

        if (total / NUM_OF_TESTS >= 70) {
            courseGrade = "Pass";
        } else {
            courseGrade = "No Pass";
        }
    }
}
```

Both subclasses **override** `computeCourseGrade()` with their own implementation. The logic is nearly identical — total the three tests, divide by `NUM_OF_TESTS` — and only the passing threshold (80 vs 70) differs. Everything *else* about a student — name, test scores, getters, setters, the constructor — is inherited untouched from `Student`. Neither subclass needs to redeclare `name`, `test`, or any of the get/set methods; they simply come along for free because of `extends Student`.

5.3 The Class Diagram



In this diagram, the `#` symbol marks the **protected** members. `Student` sits at the top as the superclass; `UndergraduateStudent` and `GraduateStudent` branch off as subclasses, each contributing only the one method they genuinely need to specialize — `computeCourseGrade()`. Every other member and method shown at the top is fully accessible to both subclasses purely through inheritance.

---

6. Using Classes Effectively with Polymorphism: The Roster Array

Now for the payoff. Because `GraduateStudent` and `UndergraduateStudent` are *both* subclasses of `Student`, a single array declared to hold `Student` references can transparently store a mix of both:

```
roster
  [0] → :GraduateStudent
  [1] → :UndergraduateStudent
  [2] → :UndergraduateStudent
  [3] → :GraduateStudent
  ...
  [39]
```

Each slot in the `roster` array is declared as type `Student`, yet it can point to *either* a `GraduateStudent` object or an `UndergraduateStudent` object — exactly the same "superclass reference, subclass object" rule from Section 4.5. When the program later calls `roster[i].computeCourseGrade()` in a loop, Java automatically runs the *correct* overridden version — the 80% rule for graduate students, the 70% rule for undergraduates — without the program ever needing to check "is this one graduate or undergraduate?" with an `if` statement. That automatic dispatch to the right overridden method, purely based on the object's actual type at runtime, **is polymorphism doing real work**: one interface (`computeCourseGrade()`), stored in one uniform array, quietly running many different implementations.

7. Summary

Today's lecture went deep into two of the four pillars of OOP. **Inheritance** allows one class to inherit the properties — both data members and methods — of another class, established with the `extends` keyword; the `Animal` / `Dog` example showed a subclass gaining a parent's method (`eat`) for free while adding its own (`bark`). **Polymorphism** means one interface, many implementations: when a single class defines multiple methods sharing a name but differing in signature, that's overloading (the `MathDemo.add` example); when a subclass redefines an inherited method with the same name and signature but different behavior, that's overriding (`speak()` in `Cat` , `computeCourseGrade()` in `GraduateStudent` / `UndergraduateStudent`) — and overriding is what produces true runtime polymorphism. Along the way, we saw the golden reference rule (a superclass variable may point to a subclass object, never the reverse), the mandatory typecast needed to call a subclass-only method through a superclass reference, and how `protected` fields let subclasses reach into inherited data without exposing it to the outside world. The `Student` / `GraduateStudent` / `UndergraduateStudent` and roster-array example tied all of it together into a realistic, working design.

8. Practice Questions

1. Suppose `Cat` and `Dog` both `extend Pet` , and `Pet` declares a `protected` field `name` (not `private`). If a third class, `Kitten` , is written as `class Kitten extends Cat` , can `Kitten` 's methods directly access `name` ? Justify your answer using the visibility rule discussed in Section 5.1, and explain what would change if `name` had been declared `private` instead.

2. In the `Student` roster example, `computeCourseGrade()` is declared in the superclass but does nothing there. What would break — or silently misbehave — if a programmer forgot to override `computeCourseGrade()` in a new subclass, say `ExchangeStudent extends Student` , and then called `computeCourseGrade()` on an `ExchangeStudent` object? Trace through exactly which version of the method actually runs.

3. Given `Pet p = new Dog();` , explain in your own words why `p.speak()` is legal without any typecast but `p.fetch()` is not. Then describe precisely what the compiler is checking when it allows or rejects each call — is it checking the *declared* type of `p` or the *actual* object `p` refers to?

4. Design (on paper, no need to run code) a small three-level hierarchy: `Employee` → `Manager` → `SeniorManager` , where each level adds one new method. If you store a `SeniorManager` object inside an `Employee` -typed reference variable, which methods can you call directly, and which ones require typecasting? Would you need one typecast or two to reach a method that exists only in `SeniorManager` ?

5. The lecture distinguishes overloading (`MathDemo.add`) from overriding (`Cat.speak()`). Construct a short example — just a sketch of class names and method signatures is enough — where a *single* class hierarchy uses **both** overloading and overriding at the same time, and explain which parts of your design demonstrate compile-time polymorphism versus runtime polymorphism.